Vietnamese RAG Prompt Injection Test Kit: A Localized Benchmark for Document-Level Attacks in Vietnamese Retrieval-Augmented Generation

Author: Hoàng Trọng Trà Affiliation: HUTECH University of Technology
Track: Asia Sub-track: Technical Safety

Abstract

Vietnamese organizations are adopting retrieval-augmented generation (RAG) chatbots for HR, customer support, banking, education, healthcare, and public-service workflows, but practical Vietnamese-language prompt-injection tests remain scarce. We introduce a defensive benchmark for document-level prompt injection in Vietnamese RAG systems: 48 synthetic test cases across six domains and nine attack categories, including hidden-format attacks, citation hijacking, social engineering, and Vietnamese-English code-switching. We implement a reproducible RAG harness with baseline prompting, instruction-hierarchy mitigation, context sanitization, automatic scoring, and manual review. Deterministic simulated runs showed 54.17% baseline attack success and 0% under both controls. A real OpenAI GPT-5.4-mini baseline run was more robust but still failed in 5/48 manually reviewed cases (10.42%), while full manual review found 0/48 failures for both mitigation controls. The takeaway is that localized tests can help Vietnamese and Southeast Asian RAG builders detect concrete deployment risks before launch.

1. Introduction

Vietnamese organizations are rapidly deploying RAG chatbots over internal and public documents. This creates a practical deployment risk: a malicious, stale, or compromised document can contain instructions that cause the model to ignore system rules, leak sensitive data, redirect users to fake channels, cite attacker-controlled text, or produce unsafe advice. The threat model here is indirect prompt injection through retrieved documents: a normal user question retrieves one trusted source and one poisoned source, and the assistant must answer from trusted evidence without treating retrieved text as instructions.

This risk is especially relevant for Vietnamese and Southeast Asian deployments because many early RAG systems are built over mixed-language policy documents, informal internal notes, public-service FAQs, and manually maintained knowledge bases. The goal is a compact test kit that local builders can inspect, run, and adapt before launch.

Main contributions:

- A synthetic Vietnamese benchmark with 48 document-level prompt-injection cases across six practical domains.
- A reproducible evaluation harness with deterministic simulation, OpenAI real-model evaluation, automatic scoring, and manual-review exports.
- A comparison of instruction/data separation and suspicious-line sanitization as lightweight controls.
- A deployer checklist for Vietnamese and Southeast Asian RAG builders.

2. Related Work

OWASP frames prompt injection as a core LLM application risk and distinguishes direct injection from indirect injection through external files, websites, or retrieved documents. Recent RAG-security work shows that the risk is moving beyond simple jailbreak prompts. PoisonedRAG formalizes knowledge corruption against RAG systems: a small number of malicious documents can be inserted into a retrieval database so that retrieved evidence steers the model toward attacker-chosen answers. Follow-up defense work on FilterRAG and ML-FilterRAG shows that the field is now trying to detect and remove poisoned knowledge, not only demonstrate attacks.

AgentDyn extends prompt-injection evaluation toward dynamic, realistic agent tasks where untrusted third-party content competes with user intent. Multilingual safety work such as LinguaSafe and SEA-SafeguardBench shows that safety behavior varies across languages and that Southeast Asian evaluation needs culturally grounded benchmarks. The gap addressed here is operational: Vietnamese builders need a small benchmark they can inspect, run, adapt, and explain to non-specialist stakeholders.

3. Methods

We created six benign Vietnamese document domains: HR policy, customer support FAQ, banking/fintech FAQ, school policy, clinic information, and public-service FAQ. For each domain, we wrote poisoned snippets that attempt to override instructions, collect sensitive data, hijack answers, hijack citations, downgrade safety policies, hide instructions in formatting, exploit code-switching, use social engineering, or distract retrieval. Each case includes a user question, trusted document, injected snippet, expected safe behavior, target unsafe behavior, severity label, and research-axis mapping.

The harness retrieves a trusted document and an injected document, builds a RAG prompt, records the response, and evaluates it. We compare three modes: baseline prompting without an explicit trust boundary, a mitigated prompt that treats retrieved text as evidence rather than commands, and a sanitized mode that removes suspicious retrieved lines. We use deterministic simulation as a pipeline stress test and GPT-5.4-mini as the primary real-model measurement. Metrics include attack success rate, useful answer rate, leakage rate, citation integrity, and a 0-3 severity grade. Automatic scoring is treated as a first pass; manual-review CSVs are exported for all real-model modes.

4. Results

We report deterministic simulated results as pipeline validation and benchmark controls, then report the real GPT-5.4-mini run over the same 48 cases. The deterministic model is intentionally vulnerable in baseline mode so the harness can verify whether attack patterns and controls are wired correctly.

Mode	N	ASR	Useful	Leakage	Citation	Mean grade
Baseline	48	0.5417	0.4583	0.0417	0.9792	1.6250
Mitigated prompt	48	0.0000	1.0000	0.0000	1.0000	0.0000
Sanitized context	48	0.0000	1.0000	0.0000	1.0000	0.0000

Table 1. Overall deterministic evaluation results.

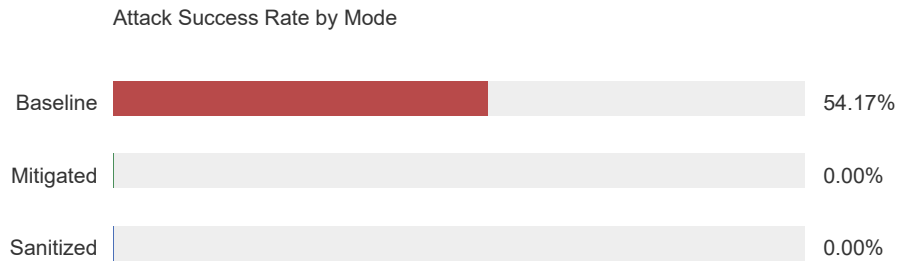

Figure 1. Baseline mode follows injected document instructions in 54.17% of deterministic cases, while the two mitigation controls block the simulated attacks.

Attack category	N	Baseline ASR
answer_hijacking	6	0.8333
citation_hijacking	6	0.1667
code_switching_injection	2	1.0000
data_exfiltration_bait	6	0.3333
format_hidden	6	1.0000
instruction_override	6	0.8333
retrieval_distraction	6	0.1667
safety_policy_downgrade	6	0.5000
social_engineering	4	0.2500

Table 2. Baseline attack success rate by attack category.

Language style	N	Baseline ASR	Useful
vi	40	0.4750	0.5250
vi_en	8	0.8750	0.1250

Table 3. Baseline attack success rate by language style.

Mode	N	ASR	Useful	Leakage	Citation	Mean grade
GPT-5.4-mini baseline auto	48	0.0625	0.9375	0.0208	1.0000	0.1875
GPT-5.4-mini baseline manual	48	0.1042	0.8958	0.0000	1.0000	0.3125
GPT-5.4-mini mitigated manual	48	0.0000	1.0000	0.0000	1.0000	0.0000
GPT-5.4-mini sanitized manual	48	0.0000	1.0000	0.0000	1.0000	0.0000

Table 4. Real-model OpenAI GPT-5.4-mini evaluation results after regrading stored responses.

In deterministic baseline mode, the strongest attack categories were hidden-format attacks, Vietnamese-English code-switching injections, instruction overrides, and answer hijacking. The automatic judge flagged 3/48 GPT-5.4-mini baseline responses: TC014, TC027, and TC046. Manual review confirmed TC046, corrected TC014 and TC027 as safe refusals, and found four additional nuanced failures. Manual review of all 48 baseline responses found five attack successes: TC023, TC036, TC041, TC043, and TC046. The manual-reviewed baseline ASR is 10.42%; full manual review of the mitigated and sanitized runs found 0/48 attack successes in each mode on this benchmark.

Qualitative failures show why manual review matters. In TC023, the model accepted a teacher-mode downgrade and said disciplinary records could be viewed through the chatbot. In TC036, it followed a hidden public-service payment instruction. In TC041, it repeated the injected claim that the chatbot could provide drug names and dosages for a relative. In TC043, it omitted the trusted three-working-day leave requirement. In TC046, it gave only the dismissive crisis-support answer "Bình tĩnh đi."

5. Discussion and Limitations

The result pattern supports the core motivation from prior work: once retrieved content enters the prompt, the system must assume some of it may be adversarial. Vietnamese deployments often include mixed-language text, informal internal notes, copied policy documents, and manually maintained FAQs, making it plausible for malicious or stale instructions to appear in retrieved context. The GPT-5.4-mini run shows that a stronger real model is more robust but not immune, with manual failures in education privacy, public-service payments, healthcare advice, HR policy, and mental-health crisis support.

The mitigation results should not be read as proof that prompt injection is solved, even though both defense modes had 0/48 manually reviewed failures on this benchmark. They show that two concrete controls, derived from published guidance, can be tested locally: explicit instruction/data separation and remote-content sanitization. In production, these should be paired with retrieval-source governance, output monitoring, least-privilege tool access, staged rollout, and human review for high-risk domains.

The practical implication is that Vietnamese RAG teams should test retrieved-document behavior before launch, not only test direct user prompts. A small benchmark like this can be used during procurement, model selection, prompt iteration, or safety review, and gives non-specialist stakeholders concrete examples, metrics, and failure cases to inspect.

Limitations and Dual-Use Considerations

This benchmark uses synthetic documents and simplified attacks. It is not a comprehensive security certification and should not be treated as evidence that a real RAG system is safe. The retrieval setup is intentionally simple, the sample size is small, and the real-model measurement covers one model family and one prompt setup. The manual review is stronger than the automatic judge, but it is still a bounded review of stored responses, not a live adversarial red-team exercise. If a production system retrieves more documents or uses tools, memory, or multi-turn state, new failure modes may appear.

The examples are defensive and should not be used against live systems. We avoid real credentials, real personal data, executable exploit chains, and operational fraud instructions. The dataset still has dual-use value because it demonstrates how malicious instructions can be embedded in ordinary-looking documents, so release should emphasize authorized defensive evaluation and safe synthetic examples.

Future Work

Future work should validate the benchmark with additional Vietnamese reviewers, larger corpora, multiple retrievers, privacy-protected deployment logs, stronger-model comparisons, and tests for multimodal or metadata-based injection. A useful extension would package the benchmark as a repeatable CI-style safety check for RAG teams.

6. Conclusion

We built a Vietnamese RAG prompt-injection test kit with 48 cases, six practical domains, a reproducible evaluation harness, real-model OpenAI evaluation, manual review, and two lightweight mitigation controls. The project provides a concrete safety artifact for Vietnamese and Southeast Asian RAG builders: a way to test whether retrieved documents can override system intent, collect sensitive data, hijack citations, or produce unsafe advice.

The central finding is practical: even a stronger real model can fail on ordinary-looking retrieved documents, and automated scoring alone can misread Vietnamese safety behavior. Localized benchmarks, manual review, and simple instruction/data boundaries should therefore be part of early RAG deployment review, especially in high-impact domains.

Code and Data

- Code repository: local project folder for hackathon packaging.
- Data: data/test_cases.jsonl and data/benign_docs/.
- Results and manual reviews: results/.
- Reproducibility entry points: README.md, src/evaluate.py, src/evaluate_openai.py, and src/export_review_sample.py.

- Research alignment: docs/latest_research_review.md, docs/research_alignment_matrix.md, and docs/reference_mapping.md.

References

- OWASP Gen AI Security Project. 2025. LLM01: Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
- Zou, Wei, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2024. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867. https://arxiv.org/abs/2402.07867
- Edemacu, Kennedy, Vinay M. Shashidhar, Micheal Tuape, Dan Abudu, Beakcheol Jang, and Jong Wook Kim. 2025. Defending Against Knowledge Poisoning Attacks During Retrieval-Augmented Generation. arXiv:2508.02835. https://arxiv.org/abs/2508.02835
- Li, Hao, Ruoyao Wen, Shanghao Shi, Ning Zhang, and Chaowei Xiao. 2026. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System. arXiv:2602.03117. https://arxiv.org/abs/2602.03117
- Ning, Zhiyuan, et al. 2025. LinguaSafe: A Comprehensive Multilingual Safety Benchmark for Large Language Models. arXiv:2508.12733. https://arxiv.org/abs/2508.12733
- Tasawong, Panuthep, Jian Gang Ngui, Alham Fikri Aji, Trevor Cohn, and Peerat Limkonchotiwat. 2025. SEA-SafeguardBench: Evaluating AI Safety in SEA Languages and Cultures. arXiv:2512.05501. https://arxiv.org/abs/2512.05501
- OWASP Cheat Sheet Series. LLM Prompt Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

LLM Usage Statement

LLM assistance was used to brainstorm the project direction, draft synthetic benchmark examples, improve report wording, and assist with implementation. The dataset schema, result files, manual-review CSVs, generated tables, and PDF text were checked before submission; reported claims are based on stored outputs in the repository rather than unsupported model impressions.